

Practical FSDP and Distributed Training State

pig7selene

September 5, 2026

Abstract

Fully Sharded Data Parallel (FSDP) makes a model whose training state would not fit on one accelerator executable by keeping parameter, gradient, and optimizer-state shards at rest and materializing a layer only when computation requires it. This chapter develops the resulting execution lifecycle, the transient memory peaks that determine whether a job fits, and the state-dictionary and checkpoint contracts needed for recovery at scale. It treats FSDP as a systems pattern rather than a framework recipe, while using established PyTorch terminology where that terminology makes the operational model precise.

1 Introduction

Chapter 10 introduced Fully Sharded Data Parallel (FSDP) as a form of Data Parallelism that replaces persistent replication with sharding. That conceptual distinction is necessary but not sufficient for operating a large training job. An FSDP configuration can have a small **steady-state** footprint and still fail because an All-Gather, a prefetched unit, a full checkpoint export, or an optimizer-state conversion creates a transient peak. Conversely, a configuration that fits can be slow because its sharding boundaries issue too many small collectives or expose communication that the backward pass cannot hide. This chapter concerns the operational state machine behind those outcomes. Its main setting is a dense model trained with a sharded Data Parallel group of size n_F . Tensor, Pipeline, and Sequence Parallelism may coexist with this group, as Chapter 10 explains, but they are not re-derived here. Likewise, the optimizer semantics and numerical-format requirements remain those of Chapters 7 and 8. The question here is narrower: when model state is distributed, which state lives where, when is a full unit reconstructed, and what must be preserved for the next update or a later restart to retain the intended semantics?

2 FSDP as an Execution Lifecycle

Let a model have P parameters, partitioned into FSDP units $U_1, \dots, U_J$. A unit is a module or explicitly chosen parameter group that will be materialized and released together. At rest, every rank holds a shard of each unit. Ignoring padding and metadata, the parameter component of one rank's persistent model state is approximately $\frac{P}{n_F}$ parameters rather than P . The same sharding principle can apply to gradients and optimizer state. This is the memory idea shared by ZeRO-style stage-3 sharding and FSDP [1], [2].

The reduction is not obtained by making a layer's computation incomplete. Before a rank can execute a local forward operation for a unit, it needs the unit's logical parameters. In a common full-shard schedule, the rank All-Gathers those parameter shards, performs the local forward computation, and releases the reconstructed parameters. Before the corresponding backward computation, it All-Gathers the unit again, forms gradients locally, then Reduce-Scatters the gradient so that each rank retains only its gradient shard. The optimizer updates local parameter and optimizer-state shards. Schematically,

sharded at rest $\rightarrow$ **All-Gather** $\rightarrow$ **local forward** $\rightarrow$ **reshard**
 $\rightarrow$ **All-Gather** $\rightarrow$ **local backward** $\rightarrow$ **Reduce-Scatter** $\rightarrow$ **local update**

The exact release points, flattening strategy, and collective implementation vary by runtime. The invariant is more important than any one interface: every operation that requires a complete unit has an explicit materialization boundary, and every state that need not remain replicated is returned to a shard after that boundary. The FSDP design documents this pattern as a practical implementation of fully sharded training [2].

2.1 All-Gather and Reduce-Scatter

An **All-Gather** concatenates or otherwise reconstructs the shards of a tensor so that every participant can access the full logical tensor. In this chapter, its usual payload is a unit’s parameters. A **Reduce-Scatter** first combines corresponding gradient contributions across ranks and then gives each rank only its assigned output shard. It therefore supplies the same averaged gradient information that replicated Data Parallelism needs while avoiding a full gradient replica after the collective.

For a unit with P_U parameters stored at b_θ bytes each, the full parameter materialization is on the order of $P_U b_\theta$ bytes per participating rank, while a persistent parameter shard is on the order of $P_U \frac{b_\theta}{n_F}$. Those expressions are accounting aids, not universal allocations: a runtime may pad flattened buffers, retain a communication workspace, use a different dtype for a temporary buffer, or overlap several units. The practical consequence is nevertheless robust. Sharding reduces state held **between** computations; it does not remove the temporary memory and network traffic required to reconstruct a unit.

Chapter 10 derives why a collective must use a consistent process group and tensor shape on all of its participants. FSDP adds a lifecycle requirement: ranks must also issue the collectives in a compatible unit order. One rank skipping a wrapped unit, taking a conditional branch that changes the sequence, or using a different wrapping plan can turn a local program error into a collective hang.

3 Units, Wrapping, and Memory Peaks

The choice of FSDP units, often called a **wrapping policy**, is a central systems decision. A unit should be large enough that its communication is not dominated by per-collective latency, but small enough that reconstructing it does not dominate the peak memory. Transformer blocks are frequently useful boundaries because they contain substantial computation and a coherent local parameter set. This is a design heuristic, not a rule: embeddings, unusually large layers, tied weights, and nested model structure can motivate different boundaries.

If one root wrapper contains the entire model, the forward pass may materialize a very large parameter set at once. If every tiny leaf is wrapped independently, the run can issue a large number of small All-Gathers and Reduce-Scatters, increasing launch overhead and making overlap difficult. Nested or block-level units create a middle ground in which the runtime can free one reconstructed unit before materializing the next. A wrapping plan is therefore a memory-and-communication schedule, not merely a way to annotate modules.

Let M_{state} denote the resident sharded parameters, gradient shards, and optimizer-state shards on one rank. A useful peak-memory decomposition is

$$M_{\text{peak}} \approx M_{\text{state}} + M_{\text{materialize}(U_{\text{current}})} + M_{\text{prefetch}} + M_{\text{act}} + M_{\text{temporary}}. \quad (1)$$

Here $M_{\text{materialize}(U_{\text{current}})}$ includes the full parameter and any transient gradient buffers needed for the current unit; M_{prefetch} is the state deliberately reconstructed ahead of use; M_{act} is the activation footprint; and $M_{\text{temporary}}$ covers kernels, communication workspaces, allocator fragmentation, and runtime bookkeeping. The first term scales roughly as

$$M_{\text{state}} \approx \frac{P}{n_F} (b_\theta + b_g + b_{\text{opt}}), \quad (2)$$

where b_{opt} is the total optimizer-state bytes per parameter. This deliberately compact model excludes activations and temporary full units; treating it as an exact device-memory prediction is a common source of failed capacity plans.

3.1 Mixed Precision, Activation Checkpointing, and Offload

FSDP composes with, rather than replaces, the precision policy from Chapter 8. A common design uses a reduced-precision parameter representation for forward and backward compute while retaining the optimizer’s update-critical state in a wider format. The bytes in Equation 2 must be counted by actual tensor role: a BF16 model copy, FP32 optimizer moments, reduced-precision gradients, and communication buffers are not interchangeable merely because they refer to the same logical parameter. A configuration should specify which dtype is used for parameters, gradient reduction, optimizer state, and accumulation.

Activation checkpointing addresses a different term in Equation 1. Instead of saving selected forward activations for backward, it saves a smaller boundary state and recomputes the omitted forward portion during backward. This exchanges extra computation for lower M_{act} ; it neither shards parameters nor makes an insufficient FSDP unit safe by itself. The general recomputation principle was formalized by Chen et al. [3]. A capacity investigation should distinguish an activation peak from a parameter-materialization peak before enabling both mechanisms indiscriminately.

CPU offload is another capacity tool, not a free memory optimization. Moving inactive parameter, gradient, or optimizer state to host memory can reduce accelerator residency, but the state must cross a host–device interconnect before use. The transfer can expose latency, contend with data loading, move optimizer computation to the host in some designs, and make an otherwise hidden communication bottleneck dominant. It is most defensible when accelerator capacity is the binding constraint and the resulting transfer schedule has been measured; official FSDP interfaces expose offload controls for this reason [4].

4 Communication Scheduling and Overlap

The execution lifecycle above contains communication that is useful only if it arrives before the corresponding computation. The step time can be summarized as

$$t_{\text{step}} \approx t_{\text{compute}} + t_{\text{uncovered-communication}} + t_{\text{recompute}} + t_{\text{serialization}}, \quad (3)$$

where $t_{\text{uncovered-communication}}$ is the portion of collective time that cannot overlap with useful computation. FSDP does not eliminate communication relative to a replicated model; it changes which tensors move and when. A job can therefore be memory-efficient but network-bound.

Forward prefetching starts an All-Gather for a future unit before its forward computation is needed. **Backward prefetching** does the analogous work for an upcoming backward unit. Both can reduce $t_{\text{uncovered-communication}}$ when the execution order is predictable and there is enough computation to hide the transfer. Both also increase M_{prefetch} , and overly aggressive prefetching can convert a communication optimization into an OOM. Current framework documentation explicitly describes this overlap–peak-memory trade-off and its dependence on a stable execution order [4].

The right order of diagnosis is usually: establish a correct non-prefetched baseline; measure collective time and rank imbalance; then introduce a bounded amount of prefetching. Do not infer success from a lower profiler value alone. The candidate must still preserve full-batch semantics, stay within peak memory under the longest sequences, and improve end-to-end valid tokens per second rather than merely shifting idle time from one stream to another.

4.1 Memory and Communication Trade-offs

Several control choices move a job along a three-way frontier rather than producing an unconditional improvement. Larger FSDP units reduce collective count but enlarge materialization peaks. More aggressive prefetching can hide network time but retains more full units. Activation checkpointing lowers activation memory but adds recomputation. CPU offload releases accelerator memory but creates host-device traffic. Higher sharding degrees reduce resident state per rank but can increase the number of communication participants and alter the effective network topology.

Control	Primary benefit	Primary cost	First measurement
Smaller FSDP units	Lower full-parameter peak and earlier release.	More collectives and less payload efficiency.	Peak memory and collective count.
Forward or backward prefetch	More communication overlap.	Additional materialized state and order sensitivity.	Uncovered communication and peak memory.
Activation checkpointing	Lower saved-activation memory.	Forward recomputation during backward.	Activation peak and step time.
CPU offload	Lower accelerator-resident state.	Host-device transfer and possible CPU-side work.	Transfer stalls and throughput.

Table 1: FSDP controls alter distinct terms in the memory and time model. A measurement should test the claimed benefit and the compensating cost together.

Table 1 also explains why copying a configuration from a model with a different block size, sequence length, network, or optimizer can fail. The useful configuration is a measured point for a declared architecture and workload, not a collection of enabled flags.

5 State Dictionaries and Distributed Checkpoints

Chapter 11 defines a resumable checkpoint as more than model weights. FSDP makes the representation of that state an additional operational choice. A **state dictionary** is a named logical view of model or optimizer state. It may be materialized in a full form, retained as shards, or produced in a rank-local form for a particular runtime. The logical model must be the same, but the memory, I/O, and portability properties of these views differ substantially.

A **full model state dictionary** reconstructs each parameter in an unsharded form. It is convenient for export, evaluation in a non-sharded environment, and interoperability with tools that expect ordinary parameter tensors. It may, however, trigger a large gathering operation and should not be silently requested on every rank. A **sharded model state dictionary** preserves distributed pieces plus enough metadata to identify the logical tensor. It is the natural scalable representation for a large training restart, provided the reader understands the shard mapping.

Optimizer state requires the same care. AdamW moments and parameter-group metadata are attached to logical parameters, while a running job may store them as local shards or flattened buffers. A checkpoint path must provide an unambiguous mapping from the saved optimizer tensors to the logical model parameters. Saving a full model state together with rank-local optimizer shards without declaring that mismatch may create an artifact that looks complete but cannot resume. The FSDP experience report emphasizes the importance of handling optimizer state alongside fully sharded model state [2].

State view	Natural use	Operational advantage	Primary risk
Full state dictionary	Export, inspection, or non-sharded loading.	Simple logical tensors for generic consumers.	Gather peak, duplicate host copies, or single-rank I/O bottleneck.
Sharded state dictionary	Large-scale save and distributed restart.	No mandatory global materialization; parallel I/O.	Requires manifest, tensor layout, and reader support.
Rank-local runtime state	Fast restart under the same controlled topology.	Can match the live storage layout closely.	Weak portability unless its mapping is made explicit.

Table 2: State-dictionary views expose the same logical training state with different materialization and recovery properties. The table is a systems taxonomy, not a promise that every framework supports every view identically.

5.1 Save, Restore, and Resharding

A distributed checkpoint writer should serialize shards in parallel, record a manifest only after all required pieces are durable, and preserve the model-state, optimizer-state, scheduler, scaler, random-state, data-state, and counter requirements from Chapter 11. The manifest must additionally identify the logical tensor names, global shapes, dtypes, shard offsets or placement, parameter-group mapping, and the parallel configuration that produced the artifact. These are not optional administrative details:

without them, a receiver cannot know whether a local shard belongs to a particular global parameter or whether an optimizer moment has the correct correspondence.

Restore should begin by constructing the intended target state and validating the checkpoint schema before training resumes. A small post-load evaluation, finite-value check, and comparison of global counters can catch a bad mapping before the first costly update. If the new run uses a different FSDP world size, a supported loader may redistribute a logical global tensor into new shards. This **resharding** is distinct from merely copying files to a different number of processes: it requires global metadata and a coordinated data movement plan. Distributed checkpoint interfaces explicitly support load-time resharding only when the saved representation and destination state make that mapping well defined [5].

Changing world size normally weakens an exact-resume claim. Random streams, data-worker assignments, collective reduction order, and local batch decomposition can differ even when model and optimizer tensors are resharded correctly. The appropriate target is usually the semantic-resume contract of Chapter 11: preserve the declared objective and update state, log the topology change, and state the reproducibility tolerance instead of implying bitwise continuity.

6 Diagnosing FSDP Failures

An FSDP OOM is meaningful only when its timing and allocation class are known. An OOM at initialization may indicate that an initial full model, a checkpoint load, or an optimizer is being materialized before sharding. An OOM just before a wrapped unit’s forward often implicates All-Gather payload, wrapping granularity, or prefetched parameters. An OOM in backward can arise from saved activations, full gradient buffers before Reduce-Scatter, optimizer workspace, or several units being retained concurrently. A late OOM during export may be a full state-dictionary gather rather than a training-step capacity failure.

Observed boundary	First scope-preserving check	Likely class of cause
Before first training step	Record which model and optimizer tensors exist before and after sharding or checkpoint load.	Initialization materialization, optimizer construction, or full-state restore.
At unit forward	Trace current and prefetched unit sizes and allocator peak.	Coarse wrapping, too much prefetch, or communication workspace.
During backward	Separate saved activations, reconstructed parameters, gradients, and recomputation buffers.	Activation peak, gradient materialization, or retained units.
Low throughput without OOM	Measure collective wait, payload sizes, and rank skew around unit boundaries.	Exposed communication, small units, topology mismatch, or load imbalance.
Hang or rank error	Compare unit order, collective shape, process groups, and conditional execution across ranks.	Collective-order divergence or incompatible sharding plan.
Resume discontinuity	Validate model and optimizer mappings, counters, and sharding metadata before the next update.	Incomplete state dictionary, corrupt shard, or unsupported resharding.

Table 3: FSDP failures should be localized to a lifecycle boundary before altering a sharding policy or learning rate. The first check preserves evidence needed to identify the responsible state transition. Communication diagnosis should report more than a single aggregate bandwidth number. Measure time spent waiting for All-Gather and Reduce-Scatter, payload sizes by unit, overlap with computation, rank-to-rank variance, sequence-length distribution, and any interaction with other parallel dimensions. A long tail on one rank can leave every peer idle at the next collective. Conversely, a high communication percentage may be acceptable if the collective is mostly hidden and the end-to-end valid-token rate improves. The correct objective is a stable training step at the intended global batch and token accounting, not the isolated minimization of one kernel or one collective.

7 Implementation Contracts

The sharding contract must name the FSDP process group, unit boundaries, logical parameter ownership, parameter and gradient reduction dtypes, optimizer-state layout, and the points at which state is materialized and released. It must state how the plan composes with Data, Tensor, Pipeline, or Sequence Parallel groups, and it must make tied parameters and conditional module execution explicit. Every rank in a group must agree on the collective order, tensor shapes, and loss-normalization semantics.

The memory contract must be evaluated at the workload’s real high-water mark: maximum permitted sequence length, target microbatch, gradient-accumulation setting, prefetch configuration, activation-checkpoint policy, optimizer initialization, and checkpoint path. It should separately record resident sharded state, activation peak, full-unit peak, communication workspace, and allocator reserve. A capacity claim based only on Equation 2 is incomplete.

The recovery contract must identify whether each artifact is full, sharded, or rank-local; provide a durable manifest for every sharded tensor; map optimizer state to logical parameters; and record the model configuration, scheduler, scaler, random and sampler states, data position, consumed-token count, and parallel topology. It must test both an ordinary restart and any supported world-size change. A successful save is not evidence of a usable checkpoint until a fresh process can load it, validate the state, and execute a controlled next step.

8 Summary

FSDP reduces persistent model-state replication by storing parameter, gradient, and optimizer shards across a Data Parallel group. Its practical behavior is governed by a lifecycle: reconstruct a unit with All-Gather when local computation needs it, release it when possible, and Reduce-Scatter gradients so that optimizer updates remain local to shards. The resulting steady-state memory reduction does not remove transient full-unit, activation, workspace, and prefetch peaks.

Wrapping determines the granularity of that lifecycle; mixed precision, activation checkpointing, offload, and prefetch each change a different term in the memory–communication–compute balance. Checkpointing must expose the logical model and optimizer state with enough distributed metadata to restore or, where supported, reshard it safely. By measuring lifecycle boundaries and treating state representation as part of the training algorithm, a team can use FSDP as a controlled distributed-training system rather than as a collection of opaque memory-saving options.

References

- [1] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020. doi: 10.1109/SC41405.2020.00024.
- [2] Y. Zhao *et al.*, “PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel,” *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 3848–3860, 2023, doi: 10.14778/3611540.3611569.
- [3] T. Chen, B. Xu, C. Zhang, and C. Guestrin, “Training Deep Nets with Sublinear Memory Cost,” *arXiv preprint arXiv:1604.06174*, 2016, doi: 10.48550/arXiv.1604.06174.
- [4] PyTorch Contributors, “FullyShardedDataParallel.” 2026. [Online]. Available: <https://docs.pytorch.org/docs/stable/fsdp.html>
- [5] PyTorch Contributors, “Distributed Checkpoint.” 2026. [Online]. Available: <https://docs.pytorch.org/docs/stable/distributed.checkpoint.html>